

CS5275 – The Algorithm Designer's Toolkit
(S2 AY2025/26)

Lecture 16:

Multiplicative weights update – basic algorithms

Prediction from expert advice

There are n experts: $i \in \{1, 2, \dots, n\}$.

There are T iterations: $t \in \{1, 2, \dots, T\}$.

In each iteration:

- Each expert provides an advice.
- You decide on an advice to follow.
- You receive a payoff based on the selected advice.

Goal: Maximize the sum of payoff over all iterations.

Prediction from expert advice

There are n experts: $i \in \{1, 2, \dots, n\}$.

There are T iterations: $t \in \{1, 2, \dots, T\}$.

In each iteration:

- Each expert provides an advice.
- You decide on an advice to follow.
- You receive a payoff based on the selected advice.

Goal: Maximize the sum of payoff over all iterations.

Many real-world applications:

- Weather forecasting
- Stock market and portfolio management
- Medical diagnosis
- Sports betting / match prediction
- ...

A more specific setting

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make your prediction.

A more specific setting

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make your prediction.
- After making the prediction, the **correct answer** is revealed.
- The goal is to minimize the number of **wrong predictions** over T iterations.

A more specific setting

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make your prediction.
- After making the prediction, the **correct answer** is revealed.
- The goal is to minimize the number of **wrong predictions** over T iterations.

The correct answer can be arbitrary or even **adversarial**.

It is possible that the adversary makes your every prediction incorrect.
(because the adversary knows your algorithm)

A more specific setting

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make your prediction.
- After making the prediction, the **correct answer** is revealed.
- The goal is to minimize the number of **wrong predictions** over T iterations.

↑
We compare the performance of our algorithm with the performance of the **best expert**.

↑
The expert that makes the smallest number of mistakes.

A more specific setting

Claim: There is an algorithm that allows you to always perform nearly as good as the best expert!

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make your prediction.
- After making the prediction, the **correct answer** is revealed.
- The goal is to minimize the number of **wrong predictions** over T iterations.

↑
We compare the performance of our algorithm with the performance of the **best expert**.

↑
The expert that makes the smallest number of mistakes.

A natural approach that does not work

Stock market:

- There are n experts.
- The stock market of each day either goes **up** or **down**.
- In each day, each expert provides a prediction: **up** or **down**.
- After seeing the expert predictions, you make **your prediction**.
- After making the prediction, the **correct answer** is revealed.
- The goal is to minimize the number of **wrong predictions** over T iterations.

Go with the majority opinion among the experts.

How to fix it?

- Need a way to distinguish between the bad and good experts.

This approach does not work:

- It is possible that in every iteration, most of the experts are wrong.

Warm-up: there is a perfect expert

Suppose there exists a perfect expert who never makes a mistake.

This algorithm ensures that you make at most $\log n$ mistakes.

- Let $S_t \subseteq \{1, \dots, n\}$ be the set of experts who **never make a mistake** in the first $t - 1$ iterations.
- In each iteration t , we decide **up** or **down** by taking a majority vote over the experts in S_t .

Warm-up: there is a perfect expert

Suppose there exists a perfect expert who never makes a mistake.

This algorithm ensures that you make at most $\log n$ mistakes.

- Let $S_t \subseteq \{1, \dots, n\}$ be the set of experts who **never make a mistake** in the first $t - 1$ iterations.
- In each iteration t , we decide **up** or **down** by taking a majority vote over the experts in S_t .

Analysis:

- At the beginning, $|S_1| = n$.
- If a mistake is made in iteration t , then $|S_{t+1}| \leq \frac{|S_t|}{2}$.

Warm-up: there is a perfect expert

Suppose there exists a perfect expert who never makes a mistake.

This algorithm ensures that you make at most $\log n$ mistakes.

- Let $S_t \subseteq \{1, \dots, n\}$ be the set of experts who **never make a mistake** in the first $t - 1$ iterations.
- In each iteration t , we decide **up** or **down** by taking a majority vote over the experts in S_t .

Analysis:

- At the beginning, $|S_1| = n$.
- If a mistake is made in iteration t , then $|S_{t+1}| \leq \frac{|S_t|}{2}$.
- If m mistakes are made in the first T iterations, then $|S_{T+1}| \leq \frac{|S_1|}{2^m} = \frac{n}{2^m}$.
- Since a perfect expert exists, $\frac{n}{2^m} \geq |S_{T+1}| \geq 1$, so **$m \leq \log n$** .

Warm-up: there is a perfect expert

Suppose there exists a perfect expert who never makes a mistake.

This algorithm ensures that you make at most $\log n$ mistakes.

The performance is nearly as good as the best expert!

Next: How to achieve this for the general case (no perfect expert)?

Multiplicative weights update: algorithm

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.

Idea: To extend the algorithm to the general case, we use weights to capture the past performance of each expert.

Multiplicative weights update: algorithm

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Idea: To extend the algorithm to the general case, we use weights to capture the past performance of each expert.

Multiplicative weights update: analysis

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Multiplicative weights update: analysis

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

Observations

$$\Phi^{(T+1)} \geq w_i^{(T+1)} = (1 - \eta)^{M_i}$$

$$\Phi^{(1)} = n$$

If a mistake is made in iteration t , then $\Phi^{(t+1)} \leq \frac{\Phi^{(t)}}{2} + \frac{\Phi^{(t)}}{2}(1 - \eta) = \Phi^{(t)} \cdot \left(1 - \frac{\eta}{2}\right)$.

Multiplicative weights update: analysis

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

$$(1 - \eta)^{M_i} \leq \Phi^{(T+1)} \leq \Phi^{(1)} \cdot \left(1 - \frac{\eta}{2}\right)^M = n \cdot \left(1 - \frac{\eta}{2}\right)^M$$

$\uparrow$ $\uparrow$ $\uparrow$
 $\Phi^{(T+1)} \geq w_i^{(T+1)} = (1 - \eta)^{M_i}$ $\Phi^{(1)} = n$

If a mistake is made in iteration t , then $\Phi^{(t+1)} \leq \frac{\Phi^{(t)}}{2} + \frac{\Phi^{(t)}}{2}(1 - \eta) = \Phi^{(t)} \cdot \left(1 - \frac{\eta}{2}\right)$.

Multiplicative weights update: analysis

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

$$(1 - \eta)^{M_i} \leq \Phi^{(T+1)} \leq \Phi^{(1)} \cdot \left(1 - \frac{\eta}{2}\right)^M = n \cdot \left(1 - \frac{\eta}{2}\right)^M$$


$$M_i \cdot \ln(1 - \eta) \leq \ln n + M \cdot \ln\left(1 - \frac{\eta}{2}\right)$$

Multiplicative weights update: analysis

Weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** based on the **weighted majority** according to $\{w_i^{(t)}\}$.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

$$(1 - \eta)^{M_i} \leq \Phi^{(T+1)} \leq \Phi^{(1)} \cdot \left(1 - \frac{\eta}{2}\right)^M = n \cdot \left(1 - \frac{\eta}{2}\right)^M$$

$$-M_i \cdot (\eta + \eta^2) \leq M_i \cdot \ln(1 - \eta) \leq \ln n + M \cdot \ln\left(1 - \frac{\eta}{2}\right) \leq \ln n - M \cdot \frac{\eta}{2}$$

$$\left. \begin{array}{l} -x - x^2 \leq \ln(1 - x) \\ e^{-x-x^2} \leq 1 - x \end{array} \right\} \forall x \leq \frac{1}{2}$$

$$\left. \begin{array}{l} 1 - x \leq e^{-x} \\ \ln(1 - x) \leq -x \end{array} \right\} \forall x$$

Multiplicative weights update: analysis

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

$$-M_i \cdot (\eta + \eta^2) \leq M_i \cdot \ln(1 - \eta) \leq \ln n + M \cdot \ln\left(1 - \frac{\eta}{2}\right) \leq \ln n - M \cdot \frac{\eta}{2}$$

Lower bound

This 2-factor is the best possible for **deterministic algorithms**.



Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Lower bound

This 2-factor is the best possible for **deterministic algorithms**.

$$M = M_1 + M_2 \geq 2 \cdot \min\{M_1, M_2\}$$

$M_1 + M_2 = T$ { Consider two experts:

- The first expert always predict **up**.
- The second expert always predict **down**.

$M = T$ { The correct answer is chosen adversarially so that you make a mistake in all iterations.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Lower bound

This 2-factor is the best possible for **deterministic algorithms**.

Claim: $M \leq 2(1 + \eta)M_i + \frac{2 \ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Next: Overcome the lower bound using **randomness**.

- Intuitively, the worst-case scenario is when two experts have very similar weights, and you always pick the wrong one.
- With randomness, you may decide your answer by a random coin flip. This reduces the chance of making a mistake from 1 to 1/2.

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}] =$ fraction of the weights of experts predicting **up**.
 - $\Pr[\text{down}] =$ fraction of the weights of experts predicting **down**.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}] =$ fraction of the weights of experts predicting **up**.
 - $\Pr[\text{down}] =$ fraction of the weights of experts predicting **down**.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

$f^{(t)}$ = weighted fraction of experts making an incorrect prediction in iteration t

$$\mathbb{E}[M] = \sum_{t=1}^T f^{(t)}$$

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}]$ = fraction of the weights of experts predicting **up**.
 - $\Pr[\text{down}]$ = fraction of the weights of experts predicting **down**.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

$f^{(t)}$ = weighted fraction of experts making an incorrect prediction in iteration t →

$$\mathbb{E}[M] = \sum_{t=1}^T f^{(t)}$$

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}]$ = fraction of the weights of experts predicting **up**.
 - $\Pr[\text{down}]$ = fraction of the weights of experts predicting **down**.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

Observations

$$\Phi^{(T+1)} \geq w_i^{(T+1)} = (1 - \eta)^{M_i}$$

$$\Phi^{(1)} = n$$

$$\Phi^{(t+1)} = (1 - f^{(t)})\Phi^{(t)} + f^{(t)}\Phi^{(t)}(1 - \eta) = \Phi^{(t)} \cdot (1 - \eta f^{(t)}) \leq \Phi^{(t)} \cdot e^{-\eta f^{(t)}}$$

$$f^{(t)} = \text{weighted fraction of experts making an incorrect prediction in iteration } t \longrightarrow \mathbb{E}[M] = \sum_{t=1}^T f^{(t)}$$

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}] = \text{fraction of the weights of experts predicting up.}$
 - $\Pr[\text{down}] = \text{fraction of the weights of experts predicting down.}$
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

$$(1 - \eta)^{M_i} \leq \Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\eta \sum_{t=1}^T f^{(t)}} = n \cdot e^{-\eta \mathbb{E}[M]}$$

$\Phi^{(T+1)} \geq w_i^{(T+1)} = (1 - \eta)^{M_i}$
 $\Phi^{(1)} = n$

$$\Phi^{(t+1)} = (1 - f^{(t)})\Phi^{(t)} + f^{(t)}\Phi^{(t)}(1 - \eta) = \Phi^{(t)} \cdot (1 - \eta f^{(t)}) \leq \Phi^{(t)} \cdot e^{-\eta f^{(t)}}$$

$$f^{(t)} = \text{weighted fraction of experts making an incorrect prediction in iteration } t \longrightarrow \mathbb{E}[M] = \sum_{t=1}^T f^{(t)}$$

Randomized algorithm

Randomized weighted majority algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Decide **up** or **down** randomly:
 - $\Pr[\text{up}] =$ fraction of the weights of experts predicting **up**.
 - $\Pr[\text{down}] =$ fraction of the weights of experts predicting **down**.
 - For every expert i who predicts wrongly, decrease its weight for the next iteration:
 - $w_i^{(t+1)} = (1 - \eta) w_i^{(t)}$.

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

Consider the potential function:

$$\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$$

$$(1 - \eta)^{M_i} \leq \Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\eta \sum_{t=1}^T f^{(t)}} = n \cdot e^{-\eta \mathbb{E}[M]}$$

$$-M_i \cdot (\eta + \eta^2) \leq M_i \cdot \ln(1 - \eta) \leq \ln n - \mathbb{E}[M] \cdot \eta$$

$$\left. \begin{array}{l} -x - x^2 \leq \ln(1 - x) \\ e^{-x-x^2} \leq 1 - x \end{array} \right\} \forall x \leq \frac{1}{2}$$

Randomized algorithm

Claim: $\mathbb{E}[M] \leq (1 + \eta)M_i + \frac{\ln n}{\eta}$ for any $i \in [n]$.

- $M_i = \#\{\text{mistakes made by expert } i\}$
- $M = \#\{\text{mistakes made by the algorithm}\}$

$$-M_i \cdot (\eta + \eta^2) \leq M_i \cdot \ln(1 - \eta) \leq \ln n - \mathbb{E}[M] \cdot \eta$$

A more general setting

- Let $m_i^{(t)}$ be the **cost** of adapting the advice of expert i in iteration t .
- So far, we only consider the **binary setting**:
 - Wrong prediction: cost = 1.
 - Correct prediction: cost = 0.
- **Next:** A more general setting where $m_i^{(t)} \in [-1,1]$.

A more general setting

In each iteration $t \in \{1, 2, \dots, T\}$:

- You select a distribution $\mathbf{p}^{(t)}$ of experts.
 - $\mathbf{p}^{(t)} = (\Pr[1 \text{ is selected}], \Pr[2 \text{ is selected}], \dots, \Pr[n \text{ is selected}])$.

A more general setting

In each iteration $t \in \{1, 2, \dots, T\}$:

- You select a distribution $\mathbf{p}^{(t)}$ of experts.
 - $\mathbf{p}^{(t)} = (\Pr[1 \text{ is selected}], \Pr[2 \text{ is selected}], \dots, \Pr[n \text{ is selected}])$.
- After selecting $\mathbf{p}^{(t)}$, the **costs** $m_i^{(t)} \in [-1, 1]$ for all $i \in \{1, 2, \dots, n\}$ are revealed.
 - The choices of $m_i^{(t)}$ is arbitrary and can even be **adversarial**.

A more general setting

In each iteration $t \in \{1, 2, \dots, T\}$:

- You select a distribution $\mathbf{p}^{(t)}$ of experts.
 - $\mathbf{p}^{(t)} = (\Pr[1 \text{ is selected}], \Pr[2 \text{ is selected}], \dots, \Pr[n \text{ is selected}])$.
- After selecting $\mathbf{p}^{(t)}$, the **costs** $m_i^{(t)} \in [-1, 1]$ for all $i \in \{1, 2, \dots, n\}$ are revealed.
 - The choices of $m_i^{(t)}$ is arbitrary and can even be **adversarial**.
- You sample an expert $i \in \{1, 2, \dots, n\}$ from the distribution $\mathbf{p}^{(t)}$, and your cost for iteration t is $m_i^{(t)}$.
 - Your **expected cost** for iteration t is $\mathbb{E}_{i \sim \mathbf{p}^{(t)}} [m_i^{(t)}] = \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}$.
 - $\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$.

$\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$, where $m_i^{(t)} \in [-1, 1]$ is the cost for expert i in iteration t .

Algorithm

Multiplicative weights algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Let $\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$.
 - Select a distribution $\mathbf{p}^{(t)} = \left(\frac{w_1^{(t)}}{\Phi^{(t)}}, \frac{w_2^{(t)}}{\Phi^{(t)}}, \dots, \frac{w_n^{(t)}}{\Phi^{(t)}} \right)$.
 - Update the weight for every expert: $w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$.

$\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$, where $m_i^{(t)} \in [-1, 1]$ is the cost for expert i in iteration t .

Analysis

Multiplicative weights algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Let $\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$.
 - Select a distribution $\mathbf{p}^{(t)} = \left(\frac{w_1^{(t)}}{\Phi^{(t)}}, \frac{w_2^{(t)}}{\Phi^{(t)}}, \dots, \frac{w_n^{(t)}}{\Phi^{(t)}} \right)$.
 - Update the weight for every expert: $w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$.

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

the total cost if we
always follow expert i

the expected cost of the algorithm

$\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$, where $m_i^{(t)} \in [-1, 1]$ is the cost for expert i in iteration t .

Analysis

Multiplicative weights algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Let $\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$.
 - Select a distribution $\mathbf{p}^{(t)} = \left(\frac{w_1^{(t)}}{\Phi^{(t)}}, \frac{w_2^{(t)}}{\Phi^{(t)}}, \dots, \frac{w_n^{(t)}}{\Phi^{(t)}} \right)$. $p_i^{(t)} = \frac{w_i^{(t)}}{\Phi^{(t)}}$
 - Update the weight for every expert: $w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$.

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

the total cost if we
always follow expert i

the expected cost of the algorithm

$$\begin{aligned}
 & \Phi^{(t+1)} \\
 &= \sum_{i=1}^n w_i^{(t+1)} \\
 &= \sum_{i=1}^n (1 - \eta m_i^{(t)}) w_i^{(t)} \\
 &= \Phi^{(t)} - \eta \Phi^{(t)} \sum_{i=1}^n m_i^{(t)} p_i^{(t)} \\
 &= \Phi^{(t)} (1 - \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}) \\
 &\leq \Phi^{(t)} e^{-\eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}
 \end{aligned}$$

$\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$, where $m_i^{(t)} \in [-1, 1]$ is the cost for expert i in iteration t .

Analysis

Multiplicative weights algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Let $\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$.
 - Select a distribution $\mathbf{p}^{(t)} = \left(\frac{w_1^{(t)}}{\Phi^{(t)}}, \frac{w_2^{(t)}}{\Phi^{(t)}}, \dots, \frac{w_n^{(t)}}{\Phi^{(t)}} \right)$. $p_i^{(t)} = \frac{w_i^{(t)}}{\Phi^{(t)}}$
 - Update the weight for every expert: $w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$.

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

the total cost if we
always follow expert i

the expected cost of the algorithm

Upper bound: $\Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}} = n \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}$

$$\begin{aligned}
 & \Phi^{(t+1)} \\
 &= \sum_{i=1}^n w_i^{(t+1)} \\
 &= \sum_{i=1}^n (1 - \eta m_i^{(t)}) w_i^{(t)} \\
 &= \Phi^{(t)} - \eta \Phi^{(t)} \sum_{i=1}^n m_i^{(t)} p_i^{(t)} \\
 &= \Phi^{(t)} (1 - \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}) \\
 &\leq \Phi^{(t)} e^{-\eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}
 \end{aligned}$$

$\mathbf{m}^{(t)} = (m_1^{(t)}, m_2^{(t)}, \dots, m_n^{(t)})$, where $m_i^{(t)} \in [-1, 1]$ is the cost for expert i in iteration t .

Analysis

Multiplicative weights algorithm:

- Fix $0 < \eta \leq \frac{1}{2}$.
- Assign initial weights to the experts to be identical weights:
 - $w_i^{(1)} = 1$ for each expert $i \in \{1, \dots, n\}$.
- In each iteration $1 \leq t \leq T$:
 - Let $\Phi^{(t)} = \sum_{i=1}^n w_i^{(t)}$.
 - Select a distribution $\mathbf{p}^{(t)} = \left(\frac{w_1^{(t)}}{\Phi^{(t)}}, \frac{w_2^{(t)}}{\Phi^{(t)}}, \dots, \frac{w_n^{(t)}}{\Phi^{(t)}} \right)$. $p_i^{(t)} = \frac{w_i^{(t)}}{\Phi^{(t)}}$
 - Update the weight for every expert: $w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$.

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

the total cost if we
always follow expert i

the expected cost of the algorithm

Upper bound: $\Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}} = n \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}$

Lower bound: $\Phi^{(T+1)} \geq w_i^{(T+1)} = \prod_{t=1}^T (1 - \eta m_i^{(t)})$

$$\begin{aligned}
 & \Phi^{(t+1)} \\
 &= \sum_{i=1}^n w_i^{(t+1)} \\
 &= \sum_{i=1}^n (1 - \eta m_i^{(t)}) w_i^{(t)} \\
 &= \Phi^{(t)} - \eta \Phi^{(t)} \sum_{i=1}^n m_i^{(t)} p_i^{(t)} \\
 &= \Phi^{(t)} (1 - \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}) \\
 &\leq \Phi^{(t)} e^{-\eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}
 \end{aligned}$$

Analysis

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

the total cost if we
always follow expert i

the expected cost of the algorithm

The sum $\sum_{\geq 0} m_i^{(t)}$ is over
all i such that $m_i^{(t)} \geq 0$

The sum $\sum_{< 0} m_i^{(t)}$ is over
all i such that $m_i^{(t)} < 0$

Upper bound: $\Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}} = n \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}$

Lower bound: $\Phi^{(T+1)} \geq w_i^{(T+1)} = \prod_{t=1}^T (1 - \eta m_i^{(t)}) \geq (1 - \eta)^{\sum_{\geq 0} m_i^{(t)}} \cdot (1 + \eta)^{-\sum_{< 0} m_i^{(t)}}$

Analysis

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

$$\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \frac{\ln n}{\eta} + \frac{1}{\eta} \sum_{\geq 0} m_i^{(t)} \ln \left(\frac{1}{1-\eta} \right) + \frac{1}{\eta} \sum_{< 0} m_i^{(t)} \ln(1+\eta)$$

the total cost if we
always follow expert i

the expected cost of the algorithm

$$(1-\eta)^x \leq 1-\eta x \quad \text{if } x \in [0,1].$$

$$(1+\eta)^{-x} \leq 1-\eta x \quad \text{if } x \in [-1,0].$$

The sum $\sum_{\geq 0} m_i^{(t)}$ is over
all i such that $m_i^{(t)} \geq 0$

The sum $\sum_{< 0} m_i^{(t)}$ is over
all i such that $m_i^{(t)} < 0$

Upper bound: $\Phi^{(T+1)} \leq \Phi^{(1)} \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}} = n \cdot e^{-\sum_{t=1}^T \eta \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)}}$

Lower bound: $\Phi^{(T+1)} \geq w_i^{(T+1)} = \prod_{t=1}^T (1 - \eta m_i^{(t)}) \geq (1-\eta)^{\sum_{\geq 0} m_i^{(t)}} \cdot (1+\eta)^{-\sum_{< 0} m_i^{(t)}}$

Analysis

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$

$$\begin{aligned}
 \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} &\leq \frac{\ln n}{\eta} + \frac{1}{\eta} \sum_{\geq 0} m_i^{(t)} \ln \left(\frac{1}{1-\eta} \right) + \frac{1}{\eta} \sum_{< 0} m_i^{(t)} \ln(1+\eta) \\
 &\leq \frac{\ln n}{\eta} + \frac{1}{\eta} \sum_{\geq 0} m_i^{(t)} (\eta + \eta^2) + \frac{1}{\eta} \sum_{< 0} m_i^{(t)} (\eta - \eta^2) \\
 &= \frac{\ln n}{\eta} + \sum_{t=1}^T m_i^{(t)} + \eta \sum_{\geq 0} m_i^{(t)} - \eta \sum_{< 0} m_i^{(t)} \\
 &= \frac{\ln n}{\eta} + \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}|
 \end{aligned}$$

the expected cost of the algorithm

the total cost if we
always follow expert i

Updating with exponential factors

- Consider the variant of the algorithm with this update rule:

$$w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\eta m_i^{(t)}}$$

- By a similar analysis, we obtain:

$$\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T (\mathbf{m}^{(t)})^2 \cdot \mathbf{p}^{(t)} + \frac{\ln n}{\eta}$$


$$(\mathbf{m}^{(t)})^2 = \left((m_1^{(t)})^2, (m_2^{(t)})^2, \dots, (m_n^{(t)})^2 \right)$$

Updating with exponential factors

- Consider the variant of the algorithm with this update rule:

$$w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\eta m_i^{(t)}}$$

- By a similar analysis, we obtain:

Exercise:

$$\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T (\mathbf{m}^{(t)})^2 \cdot \mathbf{p}^{(t)} + \frac{\ln n}{\eta}$$

The lower bound of $\Phi^{(T+1)}$ is simpler due to the exponential update rule.

The upper bound of $\Phi^{(T+1)}$ is more complicated and requires this inequality:

$$e^{-\eta x} \leq 1 - \eta x + \eta^2 x^2 \text{ for } |\eta x| \leq 1$$


$$(\mathbf{m}^{(t)})^2 = \left((m_1^{(t)})^2, (m_2^{(t)})^2, \dots, (m_n^{(t)})^2 \right)$$

Extension 1: gains instead of losses

$$0 < \eta \leq \frac{1}{2}$$

Suppose $m_i^{(t)} \in [-1, 1]$ is the **cost** of selecting expert i in iteration t .



$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\ln n}{\eta}$$

$$w_i^{(t+1)} = (1 - \eta m_i^{(t)}) w_i^{(t)}$$

Suppose $m_i^{(t)} \in [-1, 1]$ is the **gain** of selecting expert i in iteration t .



$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \geq \sum_{t=1}^T m_i^{(t)} - \eta \sum_{t=1}^T |m_i^{(t)}| - \frac{\ln n}{\eta}$$

$$w_i^{(t+1)} = (1 + \eta m_i^{(t)}) w_i^{(t)}$$

Extension 2: range of $m_i^{(t)}$

$$0 < \eta \leq \frac{1}{2}$$

Suppose $m_i^{(t)} \in [-\rho, \rho]$ is the **cost** of selecting expert i in iteration t .

↓

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\rho \ln n}{\eta}$

 $w_i^{(t+1)} = \left(1 - \frac{\eta m_i^{(t)}}{\rho}\right) w_i^{(t)}$

Suppose $m_i^{(t)} \in [-\rho, \rho]$ is the **gain** of selecting expert i in iteration t .

↓

Claim: $\sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \geq \sum_{t=1}^T m_i^{(t)} - \eta \sum_{t=1}^T |m_i^{(t)}| - \frac{\rho \ln n}{\eta}$

 $w_i^{(t+1)} = \left(1 + \frac{\eta m_i^{(t)}}{\rho}\right) w_i^{(t)}$

$\mathbf{p}^*$ = any distribution over the set experts $\{1, 2, \dots, n\}$.

$$|\mathbf{m}^{(t)}| = (|m_1^{(t)}|, |m_2^{(t)}|, \dots, |m_n^{(t)}|)$$

Extension 3: convex combination of experts $0 < \eta \leq \frac{1}{2}$

Suppose $m_i^{(t)} \in [-\rho, \rho]$ is the **cost** of selecting expert i in iteration t .

$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \eta \sum_{t=1}^T |m_i^{(t)}| + \frac{\rho \ln n}{\eta}$$

$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \leq \sum_{t=1}^T (\mathbf{m}^{(t)} + \eta |\mathbf{m}^{(t)}|) \cdot \mathbf{p}^* + \frac{\rho \ln n}{\eta}$$

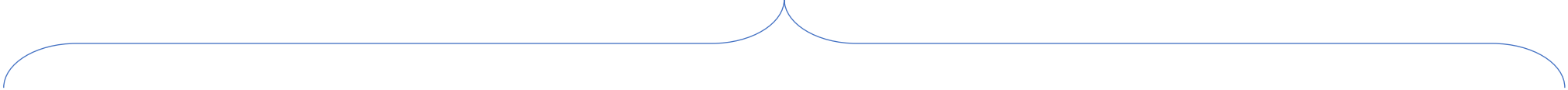
Suppose $m_i^{(t)} \in [-\rho, \rho]$ is the **gain** of selecting expert i in iteration t .

$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \geq \sum_{t=1}^T m_i^{(t)} - \eta \sum_{t=1}^T |m_i^{(t)}| - \frac{\rho \ln n}{\eta}$$

$$\text{Claim: } \sum_{t=1}^T \mathbf{m}^{(t)} \cdot \mathbf{p}^{(t)} \geq \sum_{t=1}^T (\mathbf{m}^{(t)} - \eta |\mathbf{m}^{(t)}|) \cdot \mathbf{p}^* - \frac{\rho \ln n}{\eta}$$

Outlook

Not just useful for the [Prediction From Expert Advice](#) problem



- The multiplicative weights update method lies at the core of **many** algorithms:
 - Ada Boost algorithm in machine learning
 - Algorithms for game playing studied in economics
 - Plotkin–Shmoys–Tardos algorithm for packing and covering LPs
 - Convex optimization: mirror descent, Lagrangian multipliers, and subgradient methods
 - Impagliazzo's proof of the Yao XOR lemma
 - ...

Outlook

- The multiplicative weights update method lies at the core of **many** algorithms:
 - Ada Boost algorithm in machine learning
 - Algorithms for game playing studied in economics
 - Plotkin–Shmoys–Tardos algorithm for packing and covering LPs
 - Convex optimization: mirror descent, Lagrangian multipliers, and subgradient methods
 - Impagliazzo's proof of the Yao XOR lemma
 - ...

Next: Using the multiplicative weights update method as a tool for designing approximation algorithms for optimization problems.

References

- Sections 1–2 of “The Multiplicative Weights Update Method: a Meta-Algorithm and Applications” by Sanjeev Arora, Elad Hazan, and Satyen Kale
 - <https://theoryofcomputing.org/articles/v008a006/>